53

操作系统安全简介

本章作者：**Peter Reiher（UCLA）**

53.1 引言

计算系统的安全是一个至关重要的话题，而且越来越重要。计算机安全一旦失守，就会造成大量金钱损失，许多人的生活也会因此受到伤害。对计算机系统的攻击极为常见，只要你在用计算机，几乎在任何场景下都难以避免。一般来说，计算机系统的所有组成部分都可能遭到攻击，其中任何一个部分有缺陷，都可能让攻击者有机会做你想阻止的事。但从安全的角度看，操作系统尤其重要。为什么？

首先，几乎所有东西都运行在操作系统之上。一般来说，如果你所依赖的底层软件不安全，不管它是操作系统、某个中间件还是别的什么，那么它上面的东西也会不安全。这就像在沙子上盖房子。你也许能盖出一座漂亮坚固的房子，可洪水照样能冲走它底下的地基；不管你盖得多么用心，它还是会被彻底毁掉。同样，你的应用程序本身也许没有任何安全缺陷，但如果攻击者能滥用你所依赖的底层软件，窃取你的信息、让你的程序崩溃，或者以其他方式伤害你，你为保护自己的代码所做的努力可能就白费了。

这一点对操作系统尤其重要。如果你不运行某个 Web 服务器或数据库系统，你可能不在乎它的安全；如果你不用某个中间件平台，你可能也不在乎它的安全。但每个人都要运行操作系统，而且可选的操作系统相对不多。因此，操作系统中的安全缺陷，尤其是广泛使用的操作系统中的安全缺陷，会对大量用户和大量软件产生巨大影响。

操作系统安全如此重要的另一个原因是，归根结底，我们所有的软件都依赖底层硬件的正确运行：处理器、内存和外围设备。最终是谁控制着这些硬件资源？操作系统。

回想一下你已经学过的内存管理、调度、文件系统（file system）、同步等内容：如果对手能迫使操作系统的这些组件随意做出任何坏事，会发生什么？如果你理解了到目前为止学到的东西，就会觉得这种前景令人深感不安[1]。我们的计算生活依赖于操作系统按规定行事，尤其依赖于它不去做那些有利于对手、不利于我们的事。

保护操作系统的安全并非易事，因为现代操作系统庞大而复杂。写代码的经验应该已经告诉你：代码越多、算法越复杂，代码里就越可能有缺陷。软件安全上的失败通常就源于这类缺陷。庞大复杂的程序往往比小而简单的程序更难保证安全，而像现代操作系统这样庞大复杂的程序并不多见。

[1]如果你不理解，那你还有很多东西要重读。很多。

保护操作系统安全的另一个难点在于，操作系统大多要同时支持多个进程（process）。你已经学过，操作系统中有许多机制，用来把进程彼此隔离（isolate），并保护共享的硬件，不让它被用来干扰其他进程。如果可以放心让每个进程对机器上的任何硬件资源和任何数据为所欲为，而不必担心它伤害其他进程，那么保护系统安全就会容易得多。然而，我们通常不会对所有东西一视同仁地信任。从一个从未访问过的网站下载一个脚本并运行，你真的希望它能删光你磁盘上的每个文件、杀掉你所有其他进程，并开始用你的网络接口向其他机器发送垃圾邮件吗？大概不希望。但如果你是这台计算机的主人，只要你愿意，你就有权做所有这些事。除非操作系统足够小心，否则它运行的任何进程，包括运行你下载的那个脚本的进程，都能做你能做的任何事。

换个角度来看操作系统安全问题。操作系统的作用之一，是为应用程序提供有用的抽象（abstraction），供它们在上面构建。这些应用程序必须依赖操作系统正确实现这些抽象，让它们按规定工作。这类抽象的定义中往往有一部分就是它们的安全行为。例如，我们期望操作系统的文件系统会实施它应当实施的访问限制。这样，应用程序就可以基于这种期望来实现所需的安全目标，比如依靠文件系统的访问保证，确保自己指定为不可写的文件不会被改动。如果应用程序不能指望操作系统抽象的安全保证得到正确实现，它们就无法利用这些抽象来实现自己的安全目标。这至少意味着应用程序开发者要多做大量工作，另想办法达到这些目标。结合我们前面的讨论，如果他们必须依赖的抽象不可信，比如虚拟内存（virtual memory）或明确定义的调度策略（scheduling policy），他们往往根本无法实现这些目标。

显然，操作系统安全至关重要，却又难以实现。那么，我们该怎样保护操作系统的安全呢？这个问题已经困扰了一代又一代的计算机科学家，至今仍没有完整的答案。不过，有一些重要的原则和工具可以用来保护操作系统的安全。你可能接触到的任何通用操作系统，一般都内置了这些原则和工具，它们决定了你能用这个系统做什么，以及该怎么做。所以，即使你觉得自己对安全不感兴趣，你也需要了解操作系统为保护自身安全做了什么，才能明白怎样让系统做你想做的事。

关键问题：如何保护操作系统资源的安全

同一台机器上运行着多个可能并发（concurrent）、相互作用的进程，在这种情况下，我们如何确保每个进程获准访问的资源恰好就是它应当访问的资源，而且访问方式恰好符合我们的要求？需要操作系统提供哪些原语？硬件应当提供哪些机制？我们如何利用它们来解决安全问题？

53.2 我们要保护什么？

我们说操作系统应当安全，那它到底要保护什么？对这一点没有相当全面的认识，就不太可能保护好。好在对操作系统来说，至少在宏观层面，这个问题很容易回答：一切。这个答案并不令人宽慰，但对操作系统安全的广泛影响，最好还是有一个现实的认识。

一个典型的通用商用操作系统完全控制着机器上的全部（或几乎全部）硬件，能够做硬件允许的任何事。这意味着它可以控制处理器，读写所有寄存器，查看任何主存位置，执行任何外设支持的任何操作。因此，操作系统能做的事情包括：

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

- 查看或修改任何进程的内存
- 读取、写入、删除或破坏任何可写持久存储（persistent storage）介质上的任何文件，包括硬盘和闪存驱动器（flash drive）上的文件
- 改变任何进程的调度，甚至停止它的执行
- 向任何地方发送任何消息，包括把进程要发送的消息篡改后再发出去
- 启用或禁用任何外围设备
- 让任何进程访问其他任何进程的资源
- 随意夺走进程控制的任何资源
- 用危害最大的谎言回应任何系统调用（system call）

本质上，进程完全受操作系统的摆布。在一个恶意的操作系统面前，进程几乎不可能“保护”自身的任何部分。我们通常假定操作系统本身并没有恶意[2]，但如果恶意进程能借某个缺陷让操作系统行为失常，后果几乎同样糟糕，因为这可能让该进程获得操作系统本身的任何能力。这应该让你认真想想设计安全的操作系统有多重要，以及（更常碰到的）给你运行的操作系统打安全补丁有多重要。操作系统中的安全缺陷可能让它所运行的机器上的一切全面失守，所以预防这类缺陷、给已发现的缺陷打补丁至关重要。

53.3 安全目标与安全策略

我们说希望操作系统（或者任何系统）是安全的，到底是什么意思？这个说法相当含糊。我们真正的意思是：有些事我们希望在系统中发生，有些事我们不希望发生，而且我们希望能高度确信自己得到了想要的结果。和生活中的大多数事情一样，得到什么，往往就要为什么付出代价，所以值得仔细想想我们真正需要哪些安全属性和效果，只为这些买单，不为用不着的东西花钱。归结起来就是：我们要明确系统中与安全相关的行为应达到哪些目标，并选择能以合理代价实现这些目标的防御方法。

安全研究人员长期以来一直在宏观层面思考这个问题，并在概念上定义了三个重要的安全目标，许多系统（包括操作系统）都有这三个目标：

- **机密性**（confidentiality）：如果某条信息应当对他人隐藏，就不要让他们知道。例如，你不希望别人知道你的信用卡号，你希望这个号码保密。
- **完整性**（integrity）：如果某条信息或系统的某个组件应当处于某个特定状态，就不要让对手改变它。例如，如果你在网上订了一份意大利辣香肠披萨外送，你不希望某个恶作剧者把你的订单改成 1000 份凤尾鱼披萨。完整性的一个重要方面是真实性（authenticity）。通常，我们不仅要确定信息没有被改动过，还要确定它是由某个特定的一方创建的，而不是由对手创建的。
- **可用性**（availability）：如果某些信息或服务应当供你自己或他人使用，就要确保攻击者无法阻止你们使用它。例如，如果你的店正在搞大促销，你不希望竞争对手能封锁你店铺周围的街道，让顾客进不了你的店。

这三个目标都有一个重要的附加维度：我们希望系统中的共享是受控的。我们会和某些人分享秘密，而不和另一些人分享。我们允许某些人修改企业的数据库，但不是随便什么人

[2]如果你怀疑你的操作系统有恶意，那就该换个操作系统了。

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

补充：安全飞地

前面我们说过，操作系统控制着机器上“几乎全部”的硬件。这种留有余地的说法应该让你忍不住要问：“那么，它不控制哪些硬件呢？”起初，它确实控制着全部硬件。但从 20 世纪 90 年代起，硬件开发者逐渐意识到，需要让某些硬件在一定程度上与操作系统隔离。最早的这类硬件主要用来保护操作系统的启动过程。**TPM**，即**可信平台模块**（Trusted Platform Module），可以保证你启动的是你想要的那个版本的操作系统，防止有人试图启动已被攻破的系统版本。近来，更通用的硬件部件开始尝试控制机器上能做什么，通常针对某些特别重要的数据，往往是与密码学（cryptography）相关的数据。这类硬件部件称为安全飞地（security enclave），因为它们的目的是只允许安全地使用这些数据，即使是系统中最强大、最受信任的代码（操作系统本身）也不例外。它们常用于支持云计算环境中的操作，在这种环境中，多个操作系统可能运行在共享同一物理硬件的虚拟机（virtual machine）里。

可实际做起来，这一招比任何人预想的都难。安全上的招数大多如此。安全飞地（security enclave）提供的隔离往往不如设计者所希望的那么彻底。不过，针对它们的攻击往往复杂而困难，而且通常要求攻击者已经能在系统上运行特权代码。所以，即使它们没有完全实现目标，也确实为防范被攻破的操作系统代码多加了一道屏障。

都行。有些系统需要对特定的一组优先用户（比如付费玩你的在线游戏的人）开放，而不对其他人（没付费的人）开放。在计算机里和在日常生活中一样，是谁在提要求，关系重大。

计算机系统安全的另一个重要方面是，我们常常希望确保某人告诉了我们某件事之后，日后不能否认他说过。这个方面通常称为**不可否认性**（non-repudiation）。否认自己的行为越难、代价越高，就越容易让人为自己的行为负责，人们也就越不可能做出恶意行为。毕竟，他们很可能会被抓住，而且很难抵赖。

这些都是宏观而笼统的目标。对一个实际系统来说，你需要深入到更详细、更具体的目标。例如，在一个典型的操作系统中，我们可能有这样一个机密性目标：一个进程的内存空间不能被另一个进程随意读取。我们可能有这样一个完整性目标：如果某个用户向某个文件写入一条记录，另一个本无权写这个文件的用户就改不了这条记录。我们可能有这样一个可用性目标：系统上运行的一个进程不能霸占 CPU，让其他进程得不到它们应得的那份 CPU。回想一下你在这门课上学到的进程抽象（process abstraction）、内存管理、调度、文件系统、进程间通信（inter-process communication，IPC）等主题，你应该能想到操作系统中我们可能需要的其他一些显而易见的机密性、完整性和可用性目标。

对任何特定的系统来说，即使是这个层次的目标也不够具体。上面提到的完整性目标（用户的文件不应被另一个无权覆盖它的用户覆盖），就提示了对特定系统而言，我们的安全目标还需要具体到什么程度。也许有某个用户应当能覆盖这个文件，比如两个人合写一份报告时就可能如此。但这并不意味着，一个没有参与这份报告的第三个用户也应当能写这个文件。我们需要能在安全目标中规定这样的细节。操作系统是写给许多需求各异的人使用的，操作系统安全也应当体现这种通用性。我们希望操作系统的安全机制能灵活地描述详细的安全目标。

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

补充：安全与容错

在讨论进程抽象（process abstraction）时，我们讲到虚拟化（virtualization）如何保护一个进程不受其他进程行为的影响。例如，我们不希望进程的内存被另一个进程意外覆盖，所以虚拟化机制必须防止这种行为。当时我们讨论的主要是进程中的缺陷或错误。这和担心恶意行为（我们讨论安全时更常见的语境）真的有什么不同吗？把资源虚拟化之后，是否就解决了所有问题？

是，也不是。（这句话是不是很有帮助？）是的，如果我们把一切都完美地虚拟化，并且不允许任何东西之间有任何交互，我们很可能已经解决了大部分恶意行为的问题。然而，大多数虚拟化机制并不是完全刀枪不入的。没有人试图破坏它们时，它们工作得很好，但面对所有可能形式的不当行为，它们未必完美。其次，或许更重要的是，我们其实并不想让进程彼此完全隔离。进程默认会共享一些操作系统资源，比如文件系统，也可以选择共享其他资源。这些有意放宽虚拟化的做法，在正确使用时并没有问题，但它们开放的合法共享，也可能成为恶意攻击的通道。最后，操作系统并不总是能完全控制硬件……

当然，最终必须由操作系统软件尽力实施这些灵活的安全目标，这意味着我们需要把这些目标编码成软件能理解的形式。我们通常必须把对安全目标的模糊理解，转换成非常具体的**安全策略**（security policy）。例如，对于上面说的那个文件，我们可能要规定这样一条策略："用户 A 和 B 可以写文件 X，其他用户都不能写。"规定得这样具体，再有精心设计和实现的机制作支撑，我们就有望实现安全目标。

注意这对操作系统安全的一个重要含义：在许多情况下，操作系统具备实现所需安全策略的机制，并且能高度可靠地正确运用它们，但前提是有人准确地告诉操作系统这条策略是什么。除了少数重要的例外（比如进程的地址空间在没有特别指示时保持私有），操作系统只提供通用的机制，用来实现各种具体的策略。然而，如果策略设计得不够明智、机制用得不够谨慎，你的操作系统实际会做的，就可能不是它应当做、也能够做的。

53.4 设计安全的系统

你们当中很少有人会自己写操作系统，对现有操作系统做重大修改的人也不多，但我们估计很多人会构建某种大型软件系统。许多计算机科学家的系统设计经验表明，有一些设计原则有助于构建有安全要求的系统。这些原则最初由 Jerome Saltzer 和 Michael Schroeder 在一篇很有影响力的论文中提出 [SS75]，不过其中有些来自其他人更早的观察。无论是原作者还是后来的评论者，都不会声称遵循这些原则就能保证系统安全，但实践证明，重视这些原则能带来更安全的系统，忽视它们则后果自负。我们在这里简要讨论一下。如果你真的要构建大型软件系统，值得去找来这篇论文（或者对它更详细的评论），仔细研究其中的概念。

1. **机制经济性**（economy of mechanism）：基本意思是让系统尽可能小、尽可能简单。简单的系统 bug 更少，行为也更容易理解。如果你不理解自己系统的行为，就不太可能知道它是否达到了安全目标。

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

2. **故障安全默认**（fail-safe defaults）：默认应当是安全的，而不是不安全的。如果可以设置策略来决定系统的行为，就让这些策略的默认设置更安全，而不是更不安全。
3. **完全仲裁**（complete mediation）：这个安全术语的意思是，每次要执行某个操作时，都应当检查该操作是否符合安全策略，每一次都要检查[3]。
4. **开放设计**（open design）：假设你的对手知道你设计的每一个细节。如果系统仍然能实现它的安全目标，你就没问题。这条原则并不一定意味着你真的要把所有细节告诉每个人，而是说你的安全措施要建立在攻击者已经知道一切的假设上。实际上，他往往确实知道。
5. **特权分离**（separation of privilege）：执行关键操作时，要求有不同的参与方或不同的凭证（credential）。例如，双因素认证（two-factor authentication）同时凭密码和你持有的某件硬件来确定身份，比单独使用其中任何一种方法都更安全。
6. **最小特权**（least privilege）：只给用户或进程执行你想允许的操作所需的最小特权。你给某一方的特权越多，它滥用这些特权的危险就越大。即使你确信这一方没有恶意，如果它犯了错，对手也可能利用它的错误，拿它多余的特权去干坏事。
7. **最少公共机制**（least common mechanism）：对不同的用户或进程，用各自独立的数据结构（data structure）或机制来处理。例如，在虚拟内存系统中，每个进程都有自己的页表（page table），从而确保一个进程无法访问另一个进程的页。
8. **可接受性**（acceptability）：这是一个关键属性，但很多程序员并不把它放在心上。如果用户不愿意用你的系统，它就一文不值。太多前景看好的安全系统被弃用，就是因为它们对用户要求太多。

关于设计安全的系统，有用的建议远不止这些。还有很多不错的资料讲的是下一步：如何把好的设计转换成能实现预期安全性的代码；另有一些资料讲的是如何评估你构建的系统是否真的达到了这些目标。这些问题超出了本课程的范围，但等到你要构建大型复杂系统时，它们就极为重要了。关于安全编程的方法，如果你用 C 语言，可以从 Seacord 的书 [SE13] 读起。如果你用的是别的语言，应当去找一本针对该语言的类似书籍，因为很多安全编码问题都和语言的细节有关。关于如何全面评估系统是否安全，可以从 Dowd 等人的著作 [D+07] 读起。

53.5 操作系统安全基础

那么，一个典型的操作系统有一组安全目标，涉及机密性、完整性和可用性的各个方面。其中一些目标往往内置在操作系统模型中，另一些则由系统的所有者或用户控制。内置的目标，是那些极其常见的目标，或者是为了让更具体的目标得以实现而必须保证的目标。这些内置目标大多与控制进程对硬件各部分的访问有关。这是因为硬件由系统上的所有进程共享，除非谨慎地控制共享，否则一个进程就可能干扰另一个进程的安全目标。其他内

[3]许多系统常常为了降低开销或提高易用性而忽视这条原则。所有工程设计都有一个压倒一切的特点：你常常必须在相互冲突的目标之间取得平衡，就像我们在课程前面（比如调度那几章）看到的那样。之后我们会在安全的语境下再多谈谈这一点。

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

提示：当心最薄弱的环节

值得记住的是，攻击你系统的人和你有许多共同的特点。特别是，他们很可能相当聪明，也很可能有点懒——这里是褒义，意思是他们不做不必要的工作。这意味着，攻击者往往会选择最容易的途径来攻破你系统的安全。如果你选了“password”作为访问系统的密码，他们是不会去找什么零日缓冲区溢出（buffer overflow）漏洞的。

对你来说，实际的含义是：在保护系统安全上花的时间，大部分应当用来找出并加固你最薄弱的环节。最薄弱的环节，就是系统中保护最差、最容易受攻击的部分，是你没法藏起来、也没法借助某种外部安全系统来加强的部分。一个运行中的系统，最薄弱的环节往往其实是它的人类用户，而不是软件。改变人的行为很难，但你可以在设计软件时考虑到，攻击者可能会试图欺骗合法用户去滥用它。还记得最小特权（least privilege）原则吗？如果攻击者能骗一个拥有全部特权的用户去滥用系统，后果会比骗一个只能损害自己资产的用户严重得多。

一般来说，思考安全和思考许多其他系统设计问题有些不同。它更具对抗性。如果想多了解怎样思考自己所构建系统的安全性，可以看看 Schneier 的书《Secrets and Lies》[SC00]。

置目标与操作系统提供的服务有关，比如文件系统、内存管理和进程间通信（interprocess communication）。如果不谨慎地控制这些服务，进程就可能破坏系统的安全目标。

显然，系统安全有很大一部分会和进程的处理有关。如果操作系统能把进程清清楚楚地分隔开，只有在操作系统的帮助下才能打破这种分隔，那么无论是共享的硬件还是操作系统服务，都不能被用来破坏我们的安全目标。这一要求意味着，操作系统在允许使用硬件和自身服务时需要非常小心。在许多情况下，操作系统也很有条件做到这一点。例如，操作系统控制着虚拟内存，而虚拟内存又完全控制着每个进程能访问哪些物理内存地址（physical memory address）。有了硬件支持，进程甚至无法引用一个没有映射到它的虚拟内存空间（virtual memory space）中的物理内存地址。（我们这些搞软件的人，应该记得经常感谢搞硬件的人给了我们这么多好东西。）

系统调用为操作系统提供了另一个实施保护的机会。在大多数操作系统中，进程要访问系统服务，就得显式地发起系统调用，这在前面的章节中讨论过。你已经学过，系统调用会把执行模式从处理器的用户模式（user mode）切换到它的管理模式（supervisor mode），同时调用一段相应的操作系统代码。这段代码可以确定是哪个进程发起了系统调用，以及该进程请求的是什么服务。前面我们只讨论了这如何让操作系统调用正确的那段系统代码来执行服务，以及在服务完成后记住该把控制权交还给谁。但同样的机制也让操作系统有机会检查：按照系统的安全策略，所请求的服务是否应当允许。由于访问外围设备要经过设备驱动程序（device driver），而设备驱动程序通常也要靠系统调用来访问，同样的机制也能保证硬件访问正确地遵循安全策略。

那么，进程发起系统调用时，操作系统会使用进程控制块（process control block）或类似结构中的进程标识符（process identifier）来确定该进程的身份。然后，操作系统可以使用**访问控制机制**（access control mechanism）来决定这个已确定身份的进程是否**获得授**

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

权（authorized），可以执行所请求的操作。如果获得了授权，操作系统要么代表该进程亲自执行这个操作，要么安排该进程自己执行，无需系统进一步干预。如果该进程没有获得授权，操作系统只需为这次系统调用生成一个错误码，并在调度算法允许时把控制权交还给该进程。

53.6 小结

操作系统的安全，无论对它自身还是对它上面的应用程序都至关重要。这种软件一旦出现安全故障，几乎可能导致无穷无尽的恶果。实现系统安全虽然不容易，但有一些已知的设计原则可以帮上忙。这些原则不仅在设计操作系统时有用，在设计任何大型软件系统时也有用。

在操作系统中实现安全，取决于你的安全目标是什么。这些目标通常涉及机密性、完整性和可用性。具体到某个系统，这些安全目标的细节各不相同，这意味着不同的系统会有不同的安全策略，帮助它们实现各自具体的安全目标。和操作系统设计的其他领域一样，我们处理这些不同需求的方式，是把任一特定系统所用的具体策略，与所有系统用来实现策略的通用机制分开。

接下来要回答的问题是：操作系统应当提供哪些机制，来帮助我们支持通用的安全策略?进程和内存的虚拟化是一种有用的机制，因为它让我们能在很大程度上控制进程的行为。在接下来的几章中，我们还会介绍其他几种有用的操作系统安全机制。

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

参考文献

[D+07] "The Art of Software Security Assessment" by Mark Dowd, John McDonald, and Justin Schuh. Addison-Wesley, 2007. *A long, comprehensive treatment of how to determine if your software system meets its security goals. It also contains useful advice on avoiding security problems in coding.*

[SC00] "Secrets and Lies" by Bruce Schneier. Wiley Computer Publishing, 2000. *A good high-level perspective of the challenges of computer security, developed at book length. Intended for an audience of moderately technically sophisticated readers, and well regarded in the security community. A must-read if you intend to work in that field.*

[SE13] "Secure Coding in C and C++" by Robert Seacord. Addison-Wesley, 2013. *A well regarded book on how to avoid major security mistakes in coding in C.*

[SS75] "The Protection of Information in Computer Systems" by Jerome Saltzer and Michael Schroeder. Proceedings of the IEEE, Vol. 63, No. 9, September 1975. *A highly influential paper, particularly their codification of principles for secure system design.*

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)